Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ОТ

Звіт

з лабораторної роботи № 5 з дисципліни
«Бази даних»
«Нормалізація бази даних»

Виконали
ІМ-51 Господінова Марія Михайлівна
ІМ-51 Резник Валерія Вадимівна

Перевірив
Хмельницький Арсеній Андрійович Київ

2026

Лабораторна робота №5. Нормалізація бази даних.

Мета:

Здійснення нормалізації реляційних відношень від початкового ненормалізованого стану до 1НФ, 2НФ та 3НФ із обґрунтуванням усунення часткових і транзитивних залежностей. Розробка SQL DDL-скриптів модифікації схеми та оновлення ER-моделі.

Завдання:

1. Формалізувати та задокументувати повний набір функціональних залежностей для всіх сутностей схеми.
2. Проаналізувати вихідні таблиці на наявність аномалій вставки (Insert), модифікації (Update) та видалення (Delete).
3. Продемонструвати поетапний процес нормалізації (1НФ - 2НФ - 3НФ).
4. Розробити DDL-скрипти (команди ALTER TABLE, CREATE TABLE) для реалізації схеми в 3НФ.
5. Оновити ER-діаграму бази даних із урахуванням нових таблиць і зв'язків.
6. Перевірити коректність функціонування оновленої структури в pgAdmin.

1. Функціональні залежності (ФЗ) початкової схеми

1.1. Таблиця users

Атрибути:

- **user_id** - первинний ключ (PK);
- **name** - ім'я користувача;
- **email** - електронна пошта (UNIQUE);
- **password_hash** - хеш пароля.

Функціональні залежності:

$$user_id \rightarrow name, email, password_hash$$

Оскільки **email** має обмеження **UNIQUE** та **NOT NULL**, він також однозначно визначає інші атрибути:

$$email \rightarrow user_id, name, password_hash$$

Кандидатні ключі:

- **user_id** (PK);
- **email** (альтернативний кандидатний ключ).

1.2. Таблиця wishlists

Атрибути:

- **wishlist_id** - первинний ключ (PK);
- **user_id** - ідентифікатор власника списку (FK);
- **name** - назва списку;
- **description** - опис списку;
- **is_public** - статус публічності.

Функціональна залежність:

$$wishlist_id \rightarrow user_id, name, description, is_public$$

Кандидатний ключ:

- **wishlist_id** (PK).

user_id не є кандидатним ключем, оскільки він може повторюватися в різних рядках таблиці wishlists. Один користувач може мати декілька списків бажань,

тому за значенням user_id неможливо однозначно визначити конкретний рядок таблиці.

1.3. Таблиця wishlist_items

Атрибути:

- **item_id** - первинний ключ (PK);
- **wishlist_id** - ідентифікатор списку (FK);
- **name** - назва товару;
- **description** - опис товару;
- **price** - ціна товару;
- **product_url** - посилання на товар;
- **image_url** - посилання на зображення;
- **is_completed** - статус виконання бажання.

Функціональна залежність:

$$item_id \rightarrow wishlist_id, name, description, price, product_url, image_url, is_completed$$

Кандидатний ключ:

- **item_id** (PK).

wishlist_id не є кандидатним ключем, оскільки він може повторюватися в різних рядках таблиці wishlist_items. Один список може містити декілька товарів, тому за значенням wishlist_id неможливо однозначно визначити конкретний рядок таблиці.

1.4. Таблиця reservations

Атрибути:

- **reservation_id** - первинний ключ (PK);
- **item_id** - ідентифікатор зарезервованого товару (FK, UNIQUE);
- **user_id** - ідентифікатор користувача, який зарезервував товар (FK);
- **reserved_at** - дата та час резервування.

Функціональні залежності:

$$reservation_id \rightarrow item_id, user_id, reserved_at$$

Оскільки **item_id** має обмеження **UNIQUE** та **NOT NULL**, він також однозначно визначає решту атрибутів:

$$item_id \rightarrow reservation_id, user_id, reserved_at$$

Кандидатні ключі:

- **reservation_id** (PK);
- **item_id** (альтернативний кандидатний ключ).

2. Аналіз надлишковості та аномалій у початковій схемі

Під час аналізу встановлено, що дані розподілені між окремими таблицями відповідно до їхнього призначення.

- **users** - зберігає інформацію про користувачів. Дані про користувача не дублюються в таблицях списків бажань або резервувань.
- **wishlists** - зберігає інформацію про списки бажань. Зв'язок із власником реалізований через user_id, тому ім'я та email користувача не потрібно зберігати повторно.
- **wishlist_items** - містить інформацію про товари. Зв'язок зі списком реалізований через **wishlist_id**, тому дані про список не дублюються в кожному товарі.
- **reservations** — зберігає інформацію про резервування. Зв'язки з товаром і користувачем реалізовані через **item_id** та **user_id**. Завдяки цьому немає потреби повторно зберігати назву товару, ім'я та email користувача.

Таким чином, у структурі початкової схеми не виявлено очевидної надлишковості, спричиненої дублюванням атрибутів між пов'язаними таблицями.

3. Покроковий процес нормалізації (1NF → 2NF → 3NF)

3.1. Перехід до першої нормальної форми (1NF)

Вимоги 1NF:

- кожен атрибут повинен містити одне атомарне значення;
- відсутні повторювані групи атрибутів;
- кожен запис повинен однозначно ідентифікуватися первинним ключем.

Перевіримо кожну таблицю.

Таблиця	Аналіз	Результат
users	Ім'я, email та хеш пароля зберігаються окремими атомарними значеннями	1NF
wishlists	Назва, опис та статус публічності зберігаються окремо	1NF
wishlist_items	Назва, опис, ціна та посилання на товар зберігаються окремими атрибутами	1NF
reservations	Ідентифікатори користувача й товару та дата резервування зберігаються окремо	1NF

Результат переходу до 1NF:

Усі чотири таблиці початкової схеми відповідають вимогам першої нормальної форми. Кожна комірка містить одне значення, повторювані групи атрибутів відсутні, а кожен рядок має первинний ключ. Тому додаткова декомпозиція таблиць для переходу до 1НФ не потрібна.

3.2. Перехід до другої нормальної форми (2NF)

Вимоги 2NF:

- таблиця повинна відповідати 1NF;
- кожен неключовий атрибут повинен повністю залежати від усього кандидатного ключа;
- не повинно бути часткових функціональних залежностей.

Перевіримо наші таблиці:

Таблиця	Кандидатний ключ	Складений ключ	Часткові залежності
users	**user_id, email**	Ні	Немає
wishlists	**wishlist_id**	Ні	Немає
wishlist_items	**item_id**	Ні	Немає
reservations	**reservation_id, item_id**	Ні	Немає

Результат переходу до 2NF:

Усі чотири таблиці відповідають вимогам другої нормальної форми. Кожна таблиця має прості кандидатні ключі, які складаються з одного атрибута. Тому часткові функціональні залежності неможливі, а всі неключові атрибути повністю залежать від відповідних кандидатних ключів. Декомпозиція таблиць для переходу до 2НФ не потрібна.

3.3. Перехід до третьої нормальної форми (3NF)

Вимоги 3NF:

- таблиця повинна відповідати 2NF;
- не повинно бути транзитивних залежностей неключових атрибутів від кандидатного ключа.

Таблиця users

Функціональні залежності:

$$user_id \rightarrow name, email, password_hash$$

$$email \rightarrow user_id, name, password_hash$$

Ім'я та хеш пароля залежать від кандидатних ключів. Інших залежностей між неключовими атрибутами не визначено.

Таблиця wishlists

Функціональна залежність:

$$wishlist_id \rightarrow user_id, name, description, is_public$$

Атрибути **name**, **description** та **is_public** визначаються ідентифікатором списку.

Інформація про власника списку зберігається в окремій таблиці **users**, а не дублюється в **wishlists**.

Таблиця wishlist_items

Функціональна залежність:

$$item_id \rightarrow wishlist_id, name, description, price, product_url, image_url, is_completed$$

Усі неключові атрибути залежать від **item_id**.

Таблиця reservations

Функціональні залежності:

$$reservation_id \rightarrow item_id, user_id, reserved_at$$

$$item_id \rightarrow reservation_id, user_id, reserved_at$$

Обидва кандидатні ключі однозначно визначають усі інші атрибути запису.

Інформація про користувача та товар зберігається в окремих таблицях, а в **reservations** використовуються зовнішні ключі.

Результат переходу до 3NF:

Усі чотири таблиці відповідають вимогам третьої нормальної форми. Не виявлено транзитивних залежностей, які вимагали б подальшої декомпозиції.

Підсумкова схема бази даних

```
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
);

CREATE TABLE wishlists (
    wishlist_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    is_public BOOLEAN NOT NULL DEFAULT TRUE,

    FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON DELETE CASCADE
);

CREATE TABLE wishlist_items (
    item_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wishlist_id UUID NOT NULL,
    name VARCHAR(150) NOT NULL,
    description TEXT,
    price NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
    product_url TEXT,
    image_url TEXT,
    is_completed BOOLEAN NOT NULL DEFAULT FALSE,

    FOREIGN KEY (wishlist_id)
        REFERENCES wishlists(wishlist_id)
        ON DELETE CASCADE
);

CREATE TABLE reservations (
    reservation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    item_id UUID NOT NULL UNIQUE,
    user_id UUID NOT NULL,
    reserved_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (item_id)
```

```
        REFERENCES wishlist_items(item_id)
        ON DELETE CASCADE,

    FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON DELETE CASCADE
);
```

Висновок

У процесі перевірки встановлено, що початкова схема Wishlist Service вже відповідає третій нормальній формі.

- На етапі 1NF підтверджено атомарність усіх атрибутів.
- На етапі 2NF не виявлено часткових функціональних залежностей.
- На етапі 3NF не виявлено транзитивних функціональних залежностей.

Тому додаткова декомпозиція таблиць не потрібна. Початкова структура з чотирма таблицями залишається без змін.